UNIVERSITÉ CÔTE D'AZUR

Chapter 4

Distributed Data Storage

Fragmentation, Replication and Fault Tolerance

Hajer AKID

Associate Professor – Université Côte d'Azur

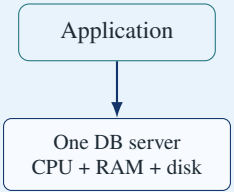
hajer.akid@univ-cotedazur.fr

M2 SLCD | 2026–2027

Why Distribute Data?

Before distribution

A single server stores the data, executes the queries, and sends results to the application.



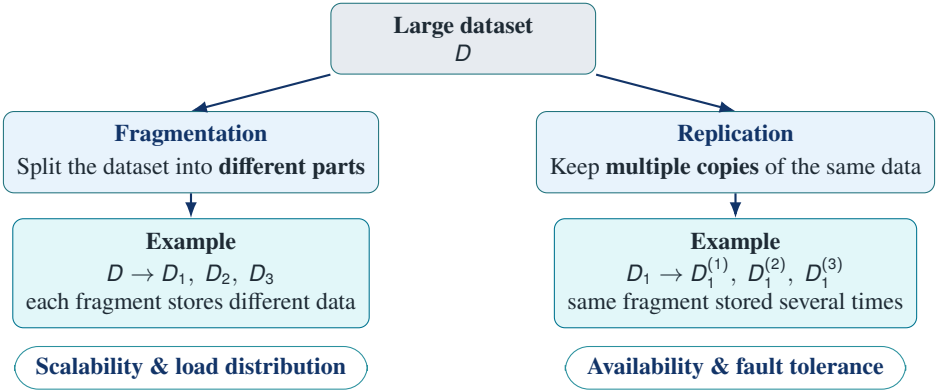
When this becomes insufficient

Data becomes difficult to manage on one machine when it exceeds:

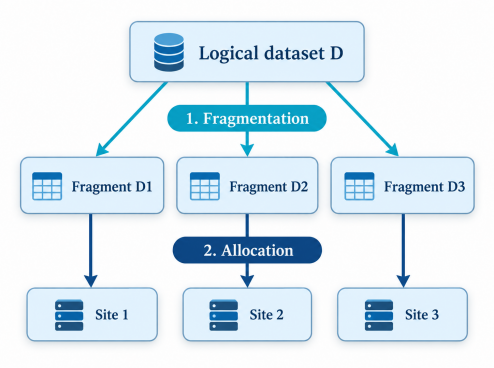
- available **RAM**;
- persistent **storage capacity**;
- acceptable **query throughput**;
- required **availability**.

Scale out: use several machines rather than relying on one bigger machine.

Two Fundamental Mechanisms



From One Dataset to Several Sites



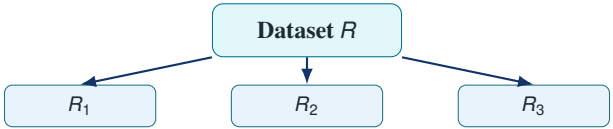
Fragmentation defines the pieces; allocation decides where the pieces are stored.

Correct Fragmentation: Three Criteria

Completeness
Every element of the original dataset must appear in at least one fragment.

Reconstructability
The original dataset must be reconstructible from all its fragments.

Disjointness
At the logical fragmentation stage, an element should normally belong to only one fragment.



Key distinction: Disjointness applies to the logical fragments; replication may later create multiple copies of a fragment.

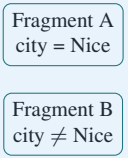
Horizontal Fragmentation

Idea

Horizontal fragmentation splits a dataset by **rows/documents/tuples**.

id	city	total
1	Nice	90
2	Paris	40
3	Nice	150
4	Lyon	80

Example



A query on `city = Nice` can be routed to a smaller fragment.

Horizontal fragmentation is the conceptual basis of many sharding techniques.

Horizontal Fragmentation: How to Distribute the Rows?

Same table structure, different rules for assigning rows to fragments.

Round-robin

Rows are assigned **one after another**.

Row 1 $\rightarrow F_1$
Row 2 $\rightarrow F_2$
Row 3 $\rightarrow F_3$
Row 4 $\rightarrow F_1$

Cyclic distribution

Hash

A hash function transforms the key into a number.

destination $= h(\text{key}) \bmod 3$

Why 3? Because we have 3 fragments.

remainder 0 $\rightarrow F_1$
remainder 1 $\rightarrow F_2$
remainder 2 $\rightarrow F_3$

Key-based distribution

Range

Values are divided into ordered intervals.

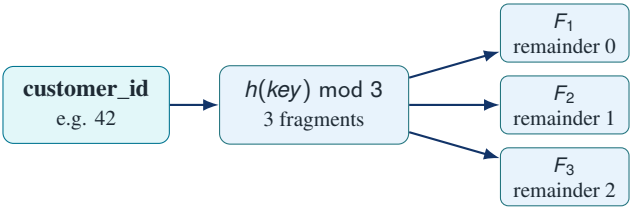
1–1000 $\rightarrow F_1$
1001–2000 $\rightarrow F_2$
2001–3000 $\rightarrow F_3$

Interval-based distribution

The rule changes, but the goal is the same: choose the destination fragment.

Hash Fragmentation: Even Distribution

The key determines the fragment through a hash function.



Why use it?

Keys tend to be spread across fragments.

Goal: balanced storage and write load.

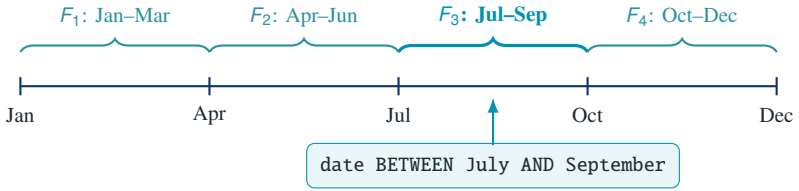
Trade-off

Nearby key values may be stored in different fragments.

Range queries become less local.

Range Fragmentation: Preserve Locality

Ordered values are grouped into contiguous ranges.



Why use it?

Nearby values remain together.

A range query can often access only the relevant fragment.

Trade-off

If new values continuously fall into the same range, that fragment may receive most writes.

Risk: hotspot.

Choosing Fragmentation Predicates

In a relational decomposition view, fragments may be defined by predicates:

```
CREATE TABLE Client_Paris AS
SELECT * FROM Client
WHERE city = 'Paris';

CREATE TABLE Client_Other AS
SELECT * FROM Client
WHERE city <> 'Paris';
```

Minimality
Avoid defining fragments that are always accessed together.

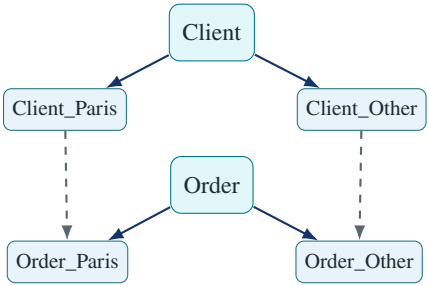
Workload-driven
Choose predicates from the most frequent and most expensive queries.

Derived Horizontal Fragmentation

Principle

If `Client` is fragmented by city, it can be useful to fragment `Order` according to the fragment of its customer.

$$\texttt{Order} \ltimes \texttt{Client_Paris}$$



Derived fragmentation co-locates related data to reduce distributed joins.

Vertical Fragmentation

Idea

Vertical fragmentation splits a relation by **attributes/columns**. A common key is kept to reconstruct the original relation.

$R_1(id, name, city)$ $R_2(id, email, phone)$

```
-- reconstructing R
SELECT R1.id, name, city, email, phone
FROM R1 JOIN R2 ON R1.id = R2.id;
```

Useful when different queries access different groups of attributes.

Vertical fragmentation reduces unnecessary attribute transfer when access patterns are attribute-specific.

Attribute Affinity

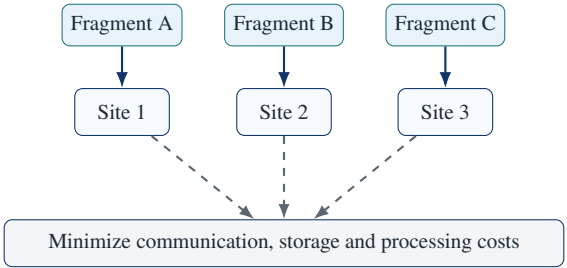
	A1	A2	A3	A4
A1	–	45	0	0
A2	45	–	5	75
A3	0	5	–	3
A4	0	75	3	–

Interpretation

a_{ij} measures how often attributes A_i and A_j are accessed together by the workload. Attributes with high affinity are good candidates to remain in the same vertical fragment.

Fragmentation is a physical design decision driven by observed or expected queries.

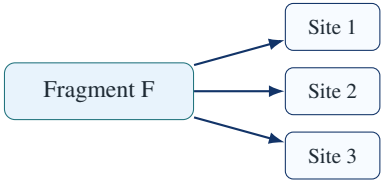
Allocation of Fragments



Non-duplicated allocation
Each fragment resides on one site.

Duplicated allocation
Some fragments are placed on several sites.

When Is Duplication Useful?

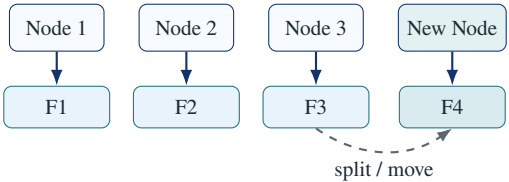


Rule of thumb: duplication helps when reads dominate updates.

Benefit
Local reads, lower latency, better availability.

Cost
Storage overhead and coherence maintenance for updates.

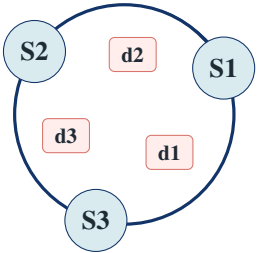
Dynamic Fragmentation and Elasticity



In real systems, the number of fragments and nodes changes over time. The system must rebalance data while minimizing pause time and data movement.

Elasticity requires fragmentation methods that can evolve.

Consistent Hashing: Intuition



Idea: hash both servers and keys on the same ring. Each key goes to the **next server clockwise**.

Why is it useful?

When a server is **added** or **removed**, only a **small part** of the key space should move.

- less data movement;
- lower rebalancing cost;
- better scalability in dynamic systems.

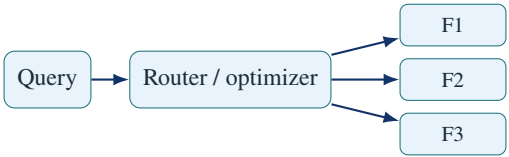
Example intuition: a new server takes mainly the keys in its local interval on the ring.

Consistent hashing minimizes redistribution when the set of servers changes.

Query Transparency

High transparency
The user writes a query without specifying fragments. The system routes, combines and optimizes.

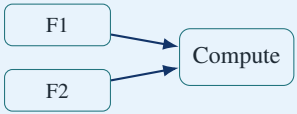
Low transparency
The user must know which fragment contains the data. This exposes naming and location complexity.



Push Query to Data vs Pull Data to Query

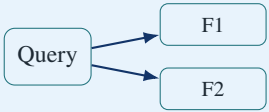
Pull data to query

Move all necessary data to one place, then compute.



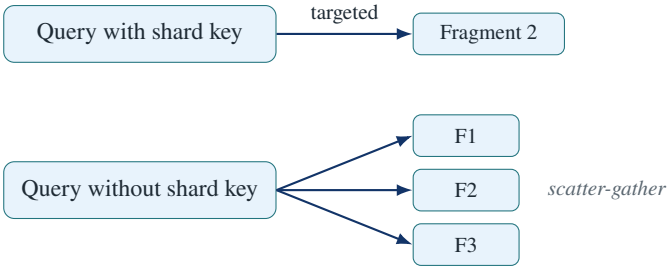
Push query to data

Send filters or partial computations to the nodes that hold the data.



In large systems, moving computation is often cheaper than moving all data.

Heuristic: Access as Few Fragments as Possible



A good fragmentation strategy lets frequent queries reach one fragment or a small number of fragments.

Replication: Why Copy Data?

Fault tolerance
If one node fails, another copy can serve the data.

Read distribution
Read traffic can be distributed across replicas.

Maintenance
Indexes or backups can be built without stopping all copies.



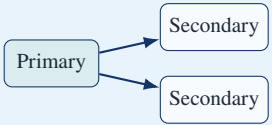
same data, several copies

Replication improves availability, but creates a consistency problem.

Replication Topologies

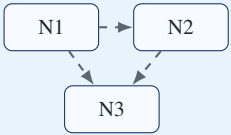
Primary-secondary

One node coordinates writes. Other nodes keep copies and may serve reads.



Multi-leader / peer

Several nodes accept writes. This can improve availability but complicates conflict resolution.

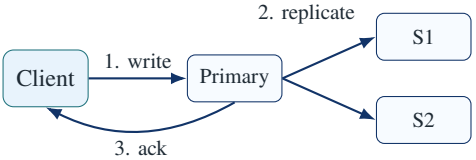


This course uses primary-secondary replication as the main model for the MongoDB case study.

Synchronous vs Asynchronous Writes

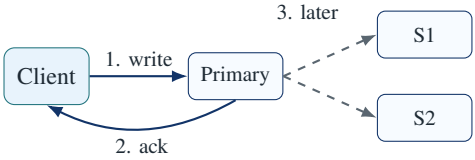
Synchronous replication

The client is acknowledged only after the required replicas have written. **Stronger consistency, higher latency.**



Asynchronous replication

The client can be acknowledged before all replicas have received the write. **Lower latency, possible temporary inconsistency.**



Consistency: What Does It Mean?

Operational view

A data system is consistent if reads reflect the writes that the application expects to have been applied. If I write value d and then read d, should I necessarily observe my own write?

Strong

Immediate visibility expected.

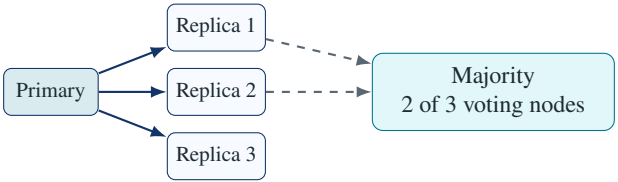
Weak

Stale reads may occur.

Eventual

Replicas converge if no new updates occur.

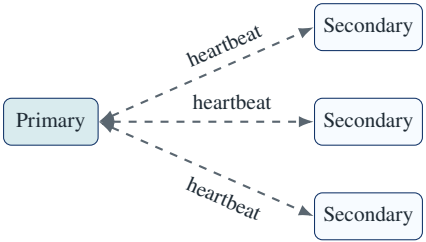
Write Acknowledgement and Quorums



A quorum is a subset large enough to avoid two independent groups making contradictory decisions. Majority-based decisions are central for leader election and failure handling.

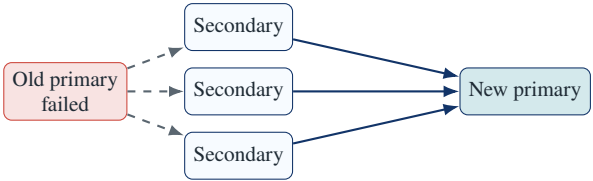
Quorums trade latency for safer coordination.

Failure Detection: Heartbeats



Distributed systems continuously exchange small messages to detect whether other nodes are alive. If a secondary fails, the system can continue. If the primary fails, a new primary must be elected.

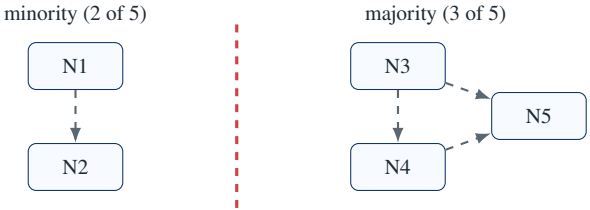
Primary Failure and Election



A majority of voting members chooses a new primary. Clients reconnect to the new primary.

Automatic election is a key mechanism for fault tolerance.

Network Partition



A network partition splits nodes into groups that cannot communicate. To avoid two primaries, only the group with a majority should be able to elect or keep a primary.

Majority prevents split-brain situations.

Restricting Transactions in Distributed Systems

Distributed transactions are expensive because they may need coordination across nodes.

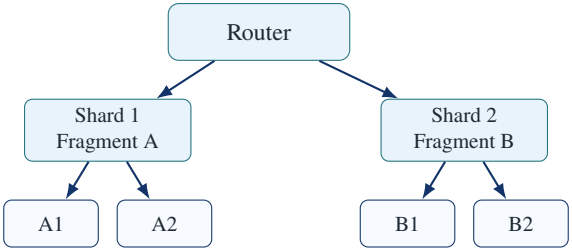
Common restrictions

- short transaction duration;
- timeout limits;
- restricted operation types;
- local access to one fragment.

Design consequence

Good physical design tries to make frequent transactions local to one fragment or one document.

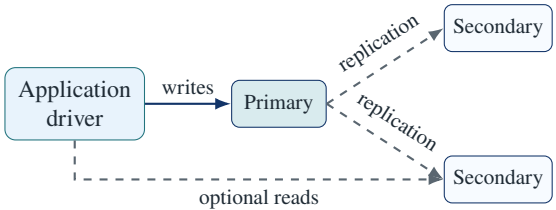
Combining Fragmentation and Replication



Each shard stores a different fragment of the dataset. Each shard can itself be replicated to tolerate failures.

This combination is the basis of scalable and fault-tolerant database clusters.

MongoDB Case Study: Replica Set

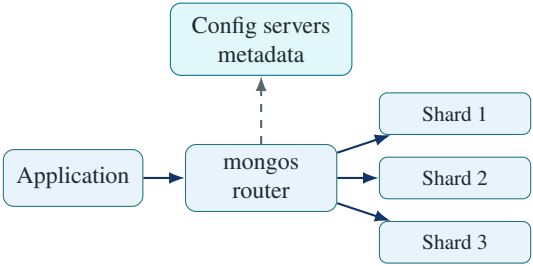


Main ideas

- writes go to the primary;
- secondaries replicate asynchronously;
- failure of the primary triggers an election;
- reads can be configured according to consistency needs.

MongoDB replica sets illustrate primary-secondary replication and failover.

MongoDB Case Study: Sharded Cluster



A sharded cluster adds a routing layer. The router uses metadata to decide which shard should receive a query. If the query does not contain the shard key, several shards may be contacted.

Shard Key: The Central Design Choice

Example

```
sh.shardCollection(
  "TDLE.users",
  { age: "hashed" }
)
```

The shard key determines how documents are distributed.

What to evaluate

- Does it distribute data evenly?
- Does it distribute writes evenly?
- Do frequent queries include it?
- Does it create hotspots?

A poor shard key can make a distributed database behave like a bottlenecked system.

Hashed Sharding vs Range Sharding

Hashed shard key

- usually good distribution;
- helps avoid monotonic-key hotspots;
- less efficient for range queries.

Range shard key

- natural for range predicates;
- can support locality;
- may create hotspots if new writes concentrate in one range.

There is no universally good key: the workload matters.

Reading a Distributed MongoDB Setup

Command family	Conceptual role
`mongod --replSet`	start a data server that belongs to a replica set
`rs.initiate()`	initialize a replica set
`rs.add(...)`	add a new replica-set member
`rs.status()`	inspect primary/secondary state and health
`mongos`	start a router for a sharded cluster
`sh.addShard(...)`	register a shard in the cluster
`sh.shardCollection(...)`	define how a collection is partitioned

These commands are not isolated syntax: each one corresponds to a distributed-systems concept.

Worked Scenario: Which Design?

Application. A retail platform stores user profiles and events. Most queries are:

- retrieve one user's profile by `user_id`;
- append a new user event;
- compute daily event counts by country;
- keep the system available if a node fails.

Candidate choices
Shard by `user_id`; replicate each shard; pre-aggregate daily counts.

Questions to ask
Will writes be balanced? Do analytical queries scatter? What can fail?

Common Misconceptions

Misconception
"Replication and sharding are the same thing." **Correction:** replication copies; sharding splits.

Misconception
"More nodes always means faster queries." **Correction:** bad routing can increase coordination and network cost.

Misconception
"A shard is just data often queried together." **Correction:** a shard is a partition determined by the sharding strategy; good keys try to support access patterns.

Misconception
"Eventual consistency means no consistency." **Correction:** it means replicas converge under stated assumptions.

Key Takeaways

1. Why distribute?
Data may exceed one machine in storage, memory, throughput or availability.

2. How to split?
Fragmentation creates logical pieces; allocation places them on sites.
Horizontal: rows/documents. **Vertical**: attributes.

3. How to remain available?
Replication creates copies, improves availability, and introduces consistency questions.

4. How does MongoDB illustrate this?
Replica sets implement replication and failover; sharded clusters distribute collections through a shard key.

Thank you!

Questions?